



GAS FAULT PREDICTION SYSTEM

VISIONARY VECTORS TEAM

TEAM MEMBERS:

TASNEEM MOHAMED

AHMAD BEHIRY

KAREEM ABDELNASSER

YOUSEF KORIEM

DATASET USED:

`synthetic_scada_pro_7days_time_ordered.csv`



Executive Summary

This project focuses on predictive maintenance for gas pipeline systems, aiming to enhance safety, reduce downtime, and optimize operational efficiency. The central objective is to predict potential system failures up to 2 hours in advance, giving engineers sufficient time to take proactive measures. To achieve this, the solution integrates real-time data streaming, machine learning predictions, and interactive visualization tools.

The approach starts by simulating SCADA-like sensor data, representing critical parameters such as pressure, temperature, flow rate, pump speed, and energy consumption. These values are continuously collected and stored in a rolling buffer equivalent to the last two hours of operation. Every 10 seconds, the system updates the buffer with the latest measurements, ensuring that the model always works with up-to-date information.

A machine learning model, previously trained on historical SCADA data with engineered features (lag values, rolling statistics, and time-based variables), is used to detect subtle patterns that may indicate early warning signs of failures. The model outputs three possible states:

- Normal – stable operation with no issues.
- Warning – anomalies detected that may lead to failure if ignored.
- Failure – high probability of a critical fault within 2 hours.

To make these insights accessible, a Streamlit dashboard was developed. The dashboard provides engineers with real-time monitoring of sensor values, prediction status, confidence levels, health indicators, and historical alerts. It also includes interactive charts for pressure, flow rate, temperature, and energy consumption, allowing users to visually track anomalies and trends over time. Auto-refreshing mechanisms ensure that the data and predictions remain current without manual intervention.

Complementing the dashboard is a chatbot interface, designed exclusively in English to serve as a smart assistant for engineers. The chatbot reads the same real-time data as the dashboard and allows engineers to ask natural-language questions about system conditions, such as pressure, energy usage, or failure predictions. This provides a fast, conversational way to query the system without navigating complex dashboards, improving usability in time-sensitive scenarios.

Overall, the project demonstrates the potential of AI-driven predictive maintenance in industrial systems. By combining data-driven forecasting, real-time visualization, and human-friendly interaction, it provides a comprehensive solution that not only predicts failures but also supports engineers in making quick, informed decisions. This proactive approach helps minimize costly downtime, ensures operational safety, and paves the way for smarter infrastructure management in gas pipeline networks.



Problem Definition

Problem: Gas system failures may cause economic losses, environmental hazards, and safety risks due to unplanned downtime or sudden breakdowns.

Goal: Predict possible failures up to 2 hours in advance so that engineers can take proactive and preventive actions, reducing risks and ensuring system reliability.

Problem: Unexpected faults in gas pipelines can lead to production interruptions, costly repairs, and potential safety incidents if not detected early.

Goal: Develop a system that can monitor real-time sensor data and predict failures in advance, enabling timely maintenance and improving overall operational safety.

Problem: Gas system malfunctions often occur suddenly, causing downtime, financial loss, and risks to human life and the environment. Traditional monitoring methods fail to provide early warnings.

Goal: Create an AI-powered predictive monitoring solution that can forecast failures up to 2 hours ahead, helping engineers mitigate risks and ensure continuous, safe operations.

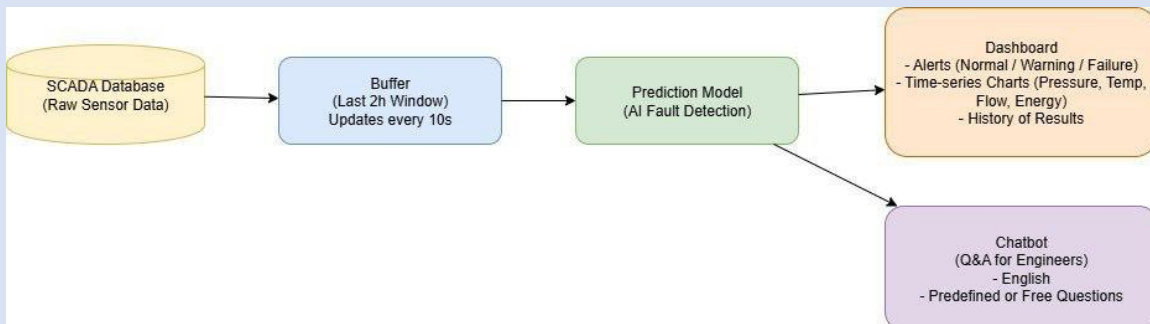
Problem: The dashboard and chatbot initially operated in isolation, leading to inconsistent outputs. The dashboard continuously generated and updated real-time predictions, while the chatbot relied on static CSV data, often returning zero or outdated values. This lack of synchronization reduced the reliability of the chatbot and limited its usefulness for engineers who depended on it for quick system queries.

Goal: Ensure real-time synchronization between the dashboard and the chatbot so that both present the same updated sensor values and predictions. Allow engineers to query the chatbot and receive results that directly reflect the latest outputs from the dashboard. Improve the consistency, accuracy, and trustworthiness of the overall system by linking both components through Google Sheets as a shared synchronization layer, after configuring the chatbot API with authentication tokens and verifying the sheet structure for correct data ordering.

Objectives

- i. Build a real-time data pipeline: Connect SCADA data → buffer (last 2 hours) → ML prediction model → Streamlit dashboard for monitoring.
- ii. Implement a data streamer: Replay historical SCADA-like data in real-time or accelerated mode, with optional Gaussian noise injection to simulate real-world sensor fluctuations.
- iii. Develop advanced ML models: Train and compare XGBoost, LightGBM, and CatBoost for fault prediction, and implement a Voting Ensemble to improve robustness and accuracy.

- iv. Feature engineering automation: Generate lag-based, rolling, and statistical features from SCADA data to capture temporal patterns leading up to system failures.
- v. Enable proactive prediction: Predict failures up to 2 hours in advance with confidence scores, allowing engineers to take preventive measures before critical breakdowns.
- vi. Design an interactive dashboard (Streamlit): Provide real-time charts (pressure, flow rate, temperature, energy consumption), system status indicators (Normal/Warning/Failure), and historical prediction logs.
- vii. Implement alerting mechanisms: Display early warning indicators, failure probability trends, and recent alerts to support proactive decision-making.
- viii. Integrate chatbot assistant: Provide an English-only chatbot for engineers to query real-time system values (pressure, temperature, energy, flow) and failure predictions in natural language.
- ix. Synchronize dashboard and chatbot results: Ensure that both visualization and conversational interfaces share the same live buffer, guaranteeing consistency across platforms.
- x. Improve usability and accessibility: Deliver a unified monitoring system where engineers can both visualize and converse with the model seamlessly, reducing cognitive load and improving trust in predictions.



System Requirements

Functional Requirements

- i. Data Ingestion: Continuously read SCADA sensor data (CSV in demo, extendable to live API/stream).
- ii. Data Buffering: Maintain a rolling buffer of the last 2 hours (≈ 720 rows if sampling every 10s).
- iii. Real-Time Updates: Update system every 10s by appending the latest data point $\rightarrow$ process buffer $\rightarrow$ generate predictions.
- iv. Prediction Engine: Pass buffered data to trained ML models (XGBoost, LightGBM, CatBoost, Ensemble) for 2-hour-ahead fault prediction.

- 
- v. Visualization: Display predictions on an interactive Streamlit dashboard including:
 - Current system status (Normal, Warning, Failure) with confidence scores.
 - Real-time sensor charts (Pressure, Flow Rate, Temperature, Energy Consumption).
 - History of predictions with timestamps, status, and confidence.
 - Alerts and trends highlighting potential failure events.
 - vi. Chatbot Integration: Provide a chatbot assistant that:
 - Answers predefined or typed English questions (e.g., "What's the current pressure?", "Is a failure predicted?").
 - Reads synchronized values directly from Google Sheets (shared with the dashboard) to ensure consistency.
 - Returns responses with contextual icons and color-coded messages.
 - vii. Simulation Support: Replay historical SCADA data with real-time pacing or accelerated mode for testing and training purposes.
 - viii. Reset and Fast Simulation Options: Allow engineers to reset data buffer or preload historical points for analysis.

Non-Functional Requirements

- i. **Performance & Reliability:**
The system must update every 10 seconds (or faster) without blocking the UI, keeping the prediction pipeline optimized for real-time operation. It should ensure resilience against missing data using auto-fill methods and provide fallback random data generation when SCADA streams or CSVs are unavailable.
- ii. **Accuracy & Robustness:**
High recall for the **Failure** class is prioritized to minimize missed alarms, with ensemble models (XGBoost, LightGBM, CatBoost) used to improve reliability and fault tolerance.
- iii. **Scalability & Maintainability:**
The architecture should easily transition from demo CSVs to live SCADA streams with minimal code changes and handle more sensors or higher-frequency data without major redesign. Components (streamer, model, dashboard, chatbot) are modular for independent upgrades, with clean code supporting easy integration of new ML models.
- iv. **Usability & Interface:**
The dashboard must be intuitive and engineer-friendly, using clear charts, alerts,

and color-coded indicators (with emojis) for quick interpretation, requiring minimal training for operators.

v. **Security & Compliance:**

If deployed in production, the system should support role-based access, optional encryption of SCADA data, and historical logging of predictions, alerts, and sensor values for auditing. Export options (CSV/Excel) should be available for offline analysis.

Dataset

Source: The project uses `synthetic_scada_pro_7days_time_ordered.csv`, a synthetic dataset generated to mimic real SCADA gas pipeline sensor readings. The data covers 7 consecutive days of operation and includes both normal and faulty scenarios.

Main Columns:

- i. timestamp – chronological record of sensor readings.
- ii. pressure – gas line pressure values.
- iii. flow_rate – flow rate through the pipeline.
- iv. temperature – temperature readings from sensors.
- v. valve_status – binary state of pipeline valves (open/closed).
- vi. pump_state – binary state of pumps (on/off).
- vii. pump_speed – RPM of pumps controlling the gas flow.
- viii. compressor_state – operational state of the compressor unit.
- ix. energy_consumption – power consumption in kWh.
- x. alarm_triggered – binary indicator if an alarm was raised.
- xi. target – system status label (0 = Normal, 1 = Warning, 2 = Failure).

Saved Files (supporting ML pipeline):

- i. feature_columns.pkl – list of engineered features used by the ML models.
- ii. preprocessor.joblib – preprocessing pipeline (scaling, encoding, transformations).
- iii. final_tuned_model.pkl – optimized ML model used for final predictions.
- iv. fault_detection_voting_model.pkl – ensemble model combining multiple classifiers for robust failure prediction.

Purpose: This dataset is used for both real-time simulation (streaming rows every 10s into the dashboard buffer) and for training/testing ML models that predict failures up to 2 hours ahead.

Data Processing & Exploratory Data Analysis (EDA)

Raw vs. Processed Data

- i. A raw copy of the dataset is always preserved in data/raw/ to ensure reproducibility.
- ii. A processed copy is maintained in data/processed/ where cleaning and feature engineering steps are applied.

Handling Missing Values

- iii. Missing values are handled using forward-fill for time-series continuity.
- iv. If still missing, fallback to median imputation for stability.

Outlier Treatment

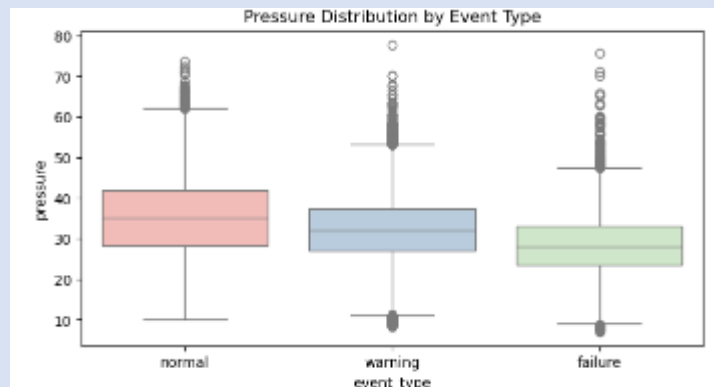
- v. Instead of deleting outliers, an additional binary column is_outlier is created.
- vi. This ensures models can learn from unusual patterns without discarding potentially important data.

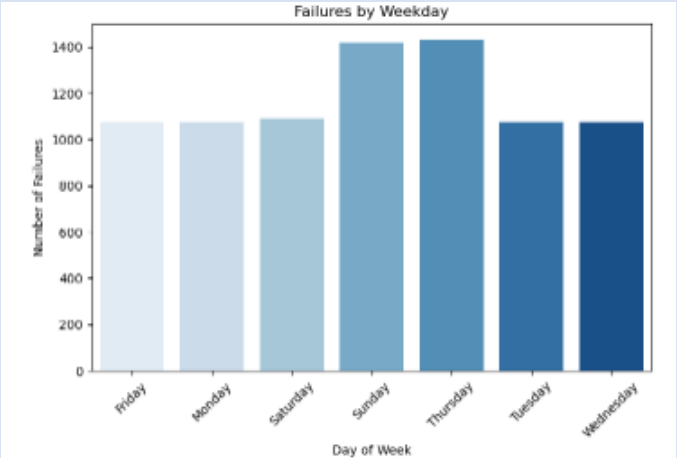
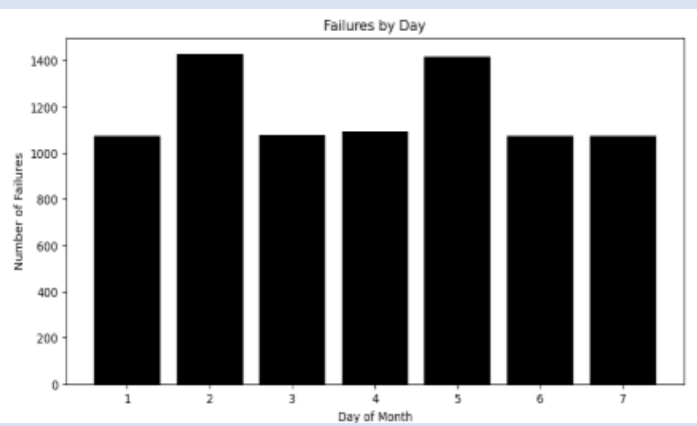
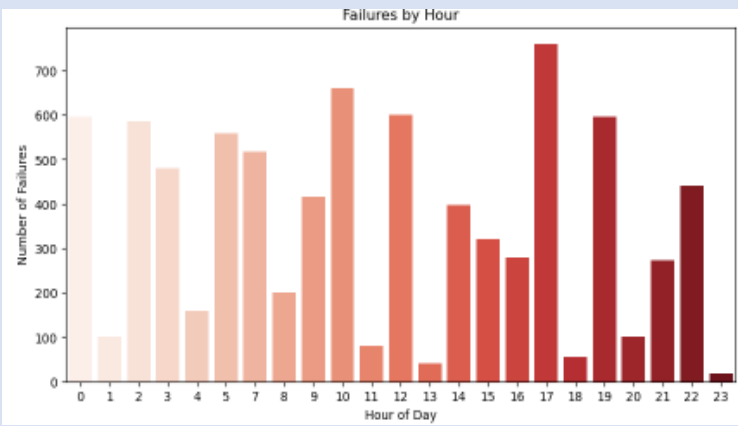
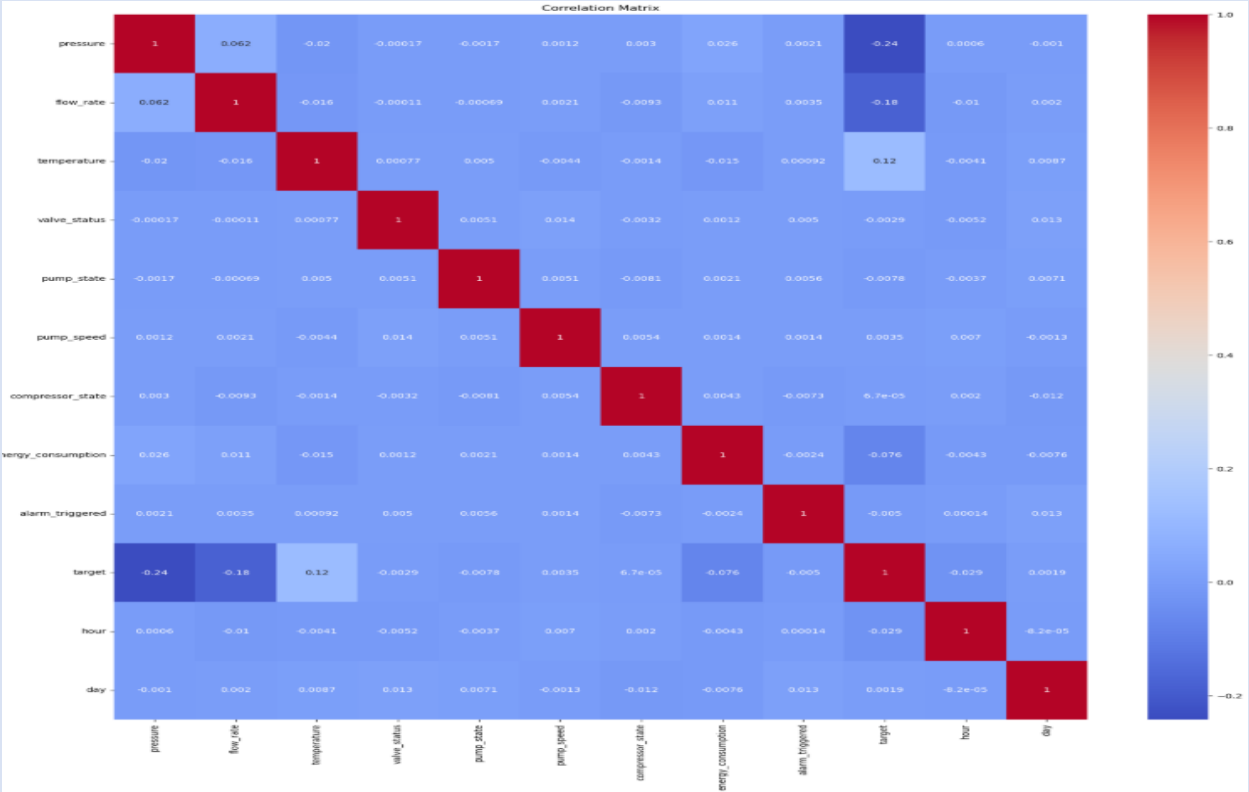
Feature Engineering

- vii. Extracted time-based features from the timestamp column:
 - hour of the day (0–23).
 - day of the month (1–31).
 - dayofweek (0=Monday, ..., 6=Sunday).
- viii. These features help the model capture daily/weekly operational cycles.

EDA Plots & Analysis

- ix. Distributions: Pressure, flow rate, temperature, and energy consumption show normal-like behavior with occasional deviations during warning/failure states.
- x. Boxplots: Highlight variations and detect sensor anomalies.
- xi. Correlation Heatmap: Identifies relationships (e.g., flow rate positively correlates with pump speed and energy consumption).
- xii. Failure-over-time plots: Visualize when failures occur, showing clusters before critical events.







Feature Engineering

To improve model performance and capture temporal dependencies, several feature engineering steps were applied:

Time Features: Extracted from the timestamp:

- hour → captures daily cycles.
- dayofweek → captures weekly operational patterns.

Lag Features: Historical values for key sensors (pressure, flow_rate, temperature, pump_speed, energy_consumption).

Lags used: t-1, t-3, t-6, t-12, t-30, t-60, t-120, t-360.

Rolling Window Statistics: For each sensor, rolling metrics were computed with windows [30, 60, 120, 360]:

- mean
- standard deviation
- minimum
- maximum

These capture short- and long-term fluctuations.

Categorical Encoding:

valve_status encoded using one-hot encoding for compatibility with ML models.

Feature Scaling:

- Applied StandardScaler for normally distributed features.
- Used RobustScaler where outliers were present.

Target Variable Creation: To align with the 2-hour prediction horizon:

- future_failure_2h = target.shift(-720)
- Assumes data is sampled every 10 seconds (720 rows $\approx$ 2 hours).

Artifacts Saved

- preprocessor.joblib → stores scaling and encoding pipeline.
- feature_columns.pkl → ensures consistent feature ordering during training and inference.

Modeling

The project experimented with several machine learning models to predict failures in the gas system.

Models Tested

- **Logistic Regression** → baseline linear classifier.
- **Random Forest** → tree-based model capturing non-linear relations.
- **XGBoost** → gradient boosting with strong performance on tabular data.
- **LightGBM** → efficient gradient boosting, tuned for speed and accuracy.
- **CatBoost** → handles categorical variables natively, robust to overfitting.

Hyperparameter Tuning

- RandomizedSearchCV applied on LightGBM for optimal depth, learning rate, and number of leaves.
- Early stopping used to prevent overfitting.

Class Imbalance Handling

- Failures are rare compared to normal operation.
- Used SMOTE (Synthetic Minority Over-sampling Technique) and class weights to improve recall for the failure class.

Ensemble Strategy

- Combined best-performing models (XGBoost + LightGBM + CatBoost) using Soft Voting.
- Ensemble improved robustness and reduced variance.

Threshold Tuning

- Adjusted decision thresholds for Class 0 (Normal) and Failure (Class 2) to minimize false negatives (missed failures).

Evaluation Metrics

- Confusion Matrix for class-wise error distribution.
- Precision, Recall, F1-score with focus on Recall for Failure detection.
- ROC-AUC to evaluate discriminatory power.

Confusion Matrix		
True Label	Normal	Fault
	4904	2051
	157	2850
Predicted Label	Normal	Fault
	12	154
Predicted Label	Normal	Fault
	1265	0

Saved Models & Artifacts

- `final_tuned_model.pkl` → optimized LightGBM model.
- `fault_detection_voting_model.pkl` → ensemble Voting Classifier.

Streaming & Inference

The project simulates real-time SCADA data flow to enable live predictions.

Simulation Setup

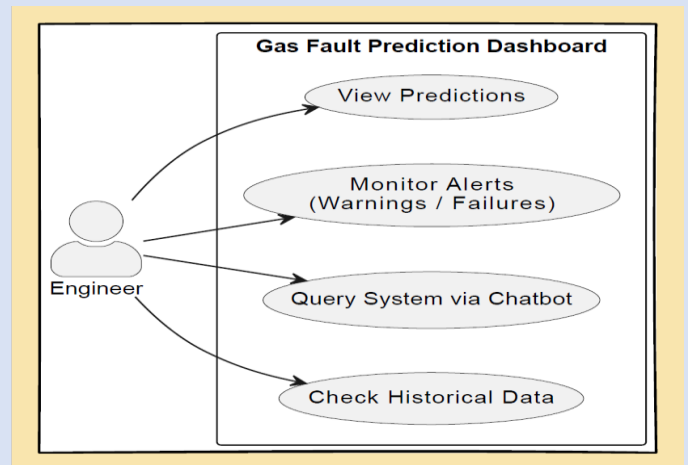
- Implemented in `streamer.py`.
- Replays rows from CSV sequentially with a 10-second step, mimicking real sensor updates.

Buffer Management

- Maintains a rolling buffer of 720 rows, representing 2 hours of historical data (assuming 10s interval).
- Oldest data automatically dropped when buffer exceeds size.

Inference Pipeline

- SCADA/CSV data replayed by streamer.
- Data stored in rolling Buffer (2h window).
- Passed through Preprocessor (feature engineering, scaling).
- Fed into ML Model (LightGBM/Ensemble).
- Model outputs Prediction (Normal, Warning, Failure).
- Predictions logged and pushed to the Dashboard.



Demo command

```
python streamer.py -csv data/synthetic_scada_pro_7days_time_ordered.csv
--step 10
python app.py
```

This ensures that the dashboard and chatbot always receive fresh, streaming predictions, similar to a real industrial SCADA environment.

Dashboard (Streamlit)

The interactive Streamlit Dashboard provides engineers with a real-time interface for monitoring, prediction, and decision support.

Sidebar Controls

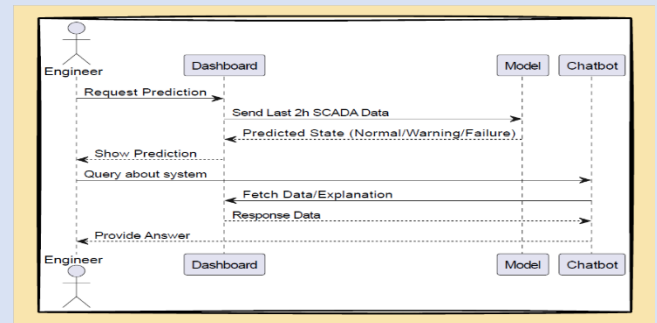
- Start / Stop the streaming process.
- Adjust replay speed (real-time / accelerated).
- Select station / scenario (normal, warning, failure).

Top KPIs (Key Performance Indicators)

- Pressure (bar).
- Flow rate (m³/s).
- Temperature (°C).
- Energy consumption (kWh).
- Current Prediction: Normal  / Warning  / Failure .

Real-Time Charts

- Time-series plots for all main sensors.
- Updated dynamically every 10 seconds.
- Interactive zoom/pan for detailed inspection.



Alerts Panel

- Displays recent warnings/failures.
- Uses color-coded indicators ( Warning /  Failure).
- Helps operators spot anomalies immediately.

History Table

- Tabular view of the latest predictions with timestamps & confidence levels.
- Export option for further offline analysis.

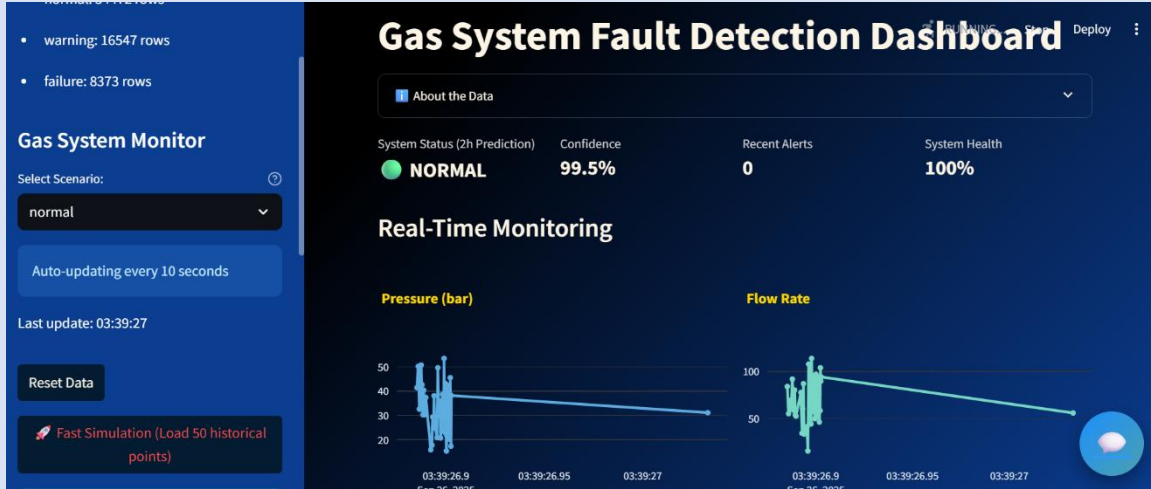
Model Insights

- Snapshot of feature importance (e.g., SHAP values).
- Shows which variables most influence the failure predictions.

Integrated Chatbot

- English-only assistant to query the system.
- Accepts both typed input and predefined questions.

- Provides quick responses about pressure, flow, temperature, energy, or failures.



Chatbot

The Smart Maintenance Assistant provides a lightweight, English-only conversational interface for engineers to query the system without navigating the dashboard manually.

Type: Rule-based chatbot (keyword-driven).

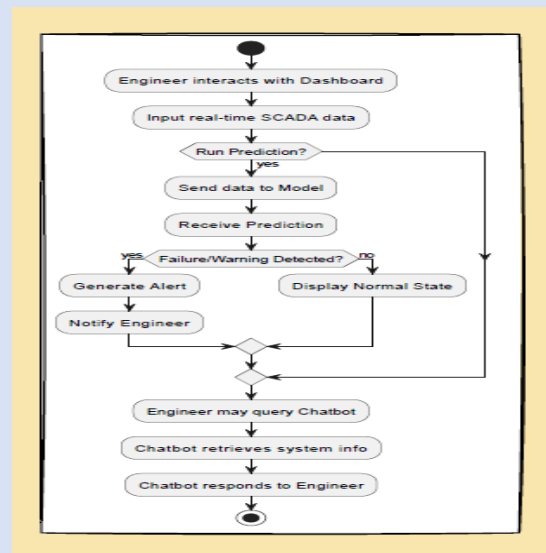
Integration: Embedded in the Streamlit dashboard with an optional floating button for quick access.

Functionality:

- Maps engineer queries to predefined keywords such as *pressure*, *temperature*, *flow*, *energy*, and *failure*.
- Retrieves the latest values directly from the real-time buffer used by the prediction model (ensuring synchronization with the dashboard).
- Displays responses in a formatted style (color-coded, icons, bold values) for easy readability.

Example Queries Supported:

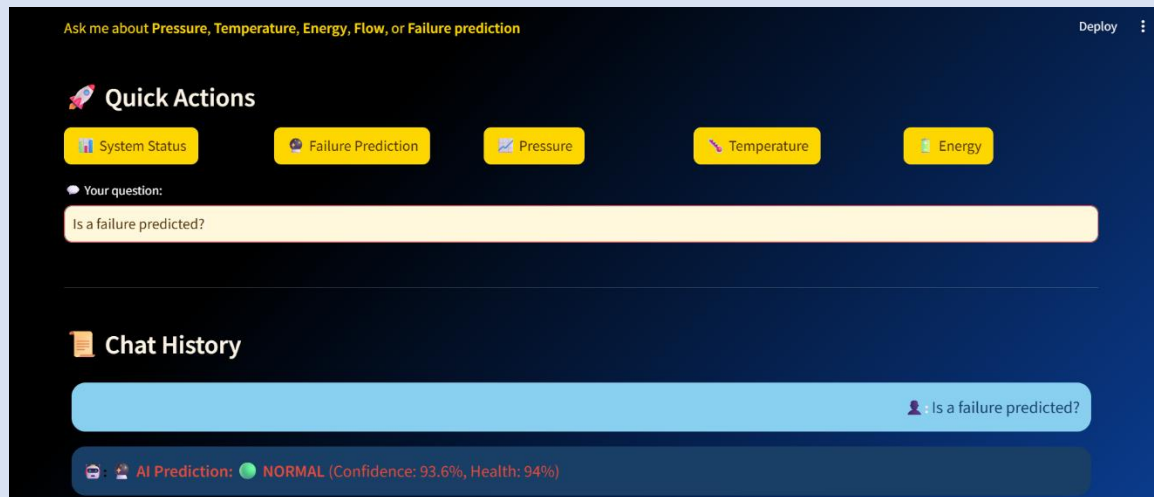
- "What is the current pressure in line 4?"
- "Is a failure predicted for station X?"
- "Show the last prediction history."
- "What was the energy consumption in the last hour?"



Key Features:

- Simple, engineer-friendly interface.
- Ensures real-time consistency with dashboard data.
- Provides quick decision support without needing to interpret charts.

The initial issue of inconsistent values between the chatbot and dashboard was resolved by integrating Google Sheets API as the synchronization layer. Authentication tokens were configured and sheet structure reviewed to ensure correct ordering of sensor values and predictions.



Database Design

In the demo implementation, no full database system was integrated; the application directly consumes CSV data streams and stores temporary results in memory buffers. However, for production deployment, a database layer is recommended to ensure persistence, traceability, and auditability of predictions and alerts.

Demo Setup:

- No dedicated DB used (CSV + in-memory session state).
- Logs and predictions are refreshed dynamically without permanent storage.

Proposed Schema (Production-Ready):

- Alerts Table Stores prediction outputs and system conditions at the time of inference.
- Logs Table Tracks user interactions, system actions, and chatbot queries.



Benefits of Database Integration:

- Long-term storage of predictions and alerts.
- Auditing & Compliance through logs.
- Ability to perform offline analytics (e.g., retraining models with historical failures).
- Scalability to real SCADA streams and integration with enterprise monitoring systems.

ERD & System Architecture

The system integrates data streaming, AI prediction, visualization, and human-machine interaction into one pipeline. The architecture can be broken into five main components:

SCADA Database (Raw Data)

- Stores historical sensor values (pressure, flow, temperature, energy, valve states).
- Acts as the data source for both training and real-time replay.

Buffer (2h Sliding Window)

- Maintains the most recent 720 rows (at 10s step = 2 hours).
- Ensures the model always receives sufficient temporal context for predictions.

Prediction Model (AI Layer)

- Ensemble of ML models (LightGBM, XGBoost, CatBoost).
- Consumes buffer → extracts engineered features → outputs probability distribution for {Normal, Warning, Failure}.
- Threshold tuning prioritizes high recall on Failure class.

Dashboard (Streamlit)

- Displays **Alerts, Charts, KPIs, and Prediction History**.
- Sidebar controls: scenario selection, fast simulation, reset.
- Provides an engineer-friendly interface with intuitive visual cues (colors, emojis).

Chatbot (English Only)

- Text-based assistant integrated into UI.
- Handles predefined + free-text queries: pressure, flow, energy, temperature, failure prediction.
- Linked to the same buffer and model outputs to ensure synchronized answers with the dashboard.

ERD (draw.io Importable)

The proposed ERD includes three main entities with relationships:

- Alerts (id, timestamp, pressure, flow_rate, temperature, valve_status, energy_consumption, prediction, probabilities, saved_at)
- Logs (id, timestamp, action, user)
- Users (id, username, role, email) [optional if authentication required]

Relations:

- One User can generate many Logs.
- Alerts are independent, but may be linked to SCADA stations if system is scaled.

The draw.io XML file can be provided separately for direct import into diagrams.net / draw.io to generate the ERD visually.

Repo Structure (GitHub)

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
├── src/
│   ├── preproc.py
│   ├── train.py
│   ├── inference.py
│   ├── streamer.py
│   └── app.py
├── models/
├── feature_columns.pkl
├── preprocessor.joblib
├── requirements.txt
├── README.md
├── presentation.pdf
└── demo_video.mp4
```




Final Submission Checklist

To ensure the project is complete and ready for submission, the following items must be included:

GitHub Repository (Public)

- Contains all project files (code, data samples, models, notebooks).
- Well-structured folders (data/, models/, notebooks/, app/).

Presentation File

- presentation.pdf (or PowerPoint exported as PDF).
- Covers Problem, Objectives, Methods, Results, Demo Plan, and Conclusion.

Demo Video

- demo_video.mp4, duration ≤ 15 minutes.
- Follows demo script (problem $\rightarrow$ EDA $\rightarrow$ pipeline $\rightarrow$ dashboard $\rightarrow$ chatbot $\rightarrow$ conclusion).

README File

- Clear run instructions:
 - `python streamer.py --csv data/... --step 10`
 - `streamlit run dashboard.py`
 - `streamlit run chatbot.py`
- Dependencies (requirements.txt).
- Overview of dataset, models, and usage.

Models & Preprocessors

- Saved in /models/:
 - `final_tuned_model.pkl`
 - `fault_detection_voting_model.pkl`
 - `feature_columns.pkl`
 - `preprocessor.joblib`

ERD / Architecture Diagram

- Provided as .png or .pdf.
- Source file (.drawio or .xml) included for edits.
- Shows pipeline: SCADA $\rightarrow$ Buffer $\rightarrow$ Model $\rightarrow$ Dashboard $\rightarrow$ Chatbot.



Appendix

Streamer Example:

```
import pandas as pd, time

from collections import deque

STEP = 10

BUFFER = 720

buffer = deque(maxlen=BUFFER)

def streamer(csv):

    df = pd.read_csv(csv, parse_dates=['timestamp'])

    for _, row in df.iterrows():

        yield row.to_dict()

    time.sleep(STEP)
```

How to Run

Run the data streamer (simulates SCADA flow)

```
python src/streamer.py
```

Start the Streamlit dashboard

```
streamlit run src/app.py
```

Notes

- STEP = 10 → means each new row is pushed every 10 seconds.
- BUFFER = 720 → keeps a rolling window of 2 hours (if 10s step).
- deque(maxlen=BUFFER) ensures only the last 720 readings are preserved.
- Synchronization was achieved via Google Sheets API. After reviewing the bot's API configuration, authentication tokens were added and sheets were verified for correct ordering of columns.